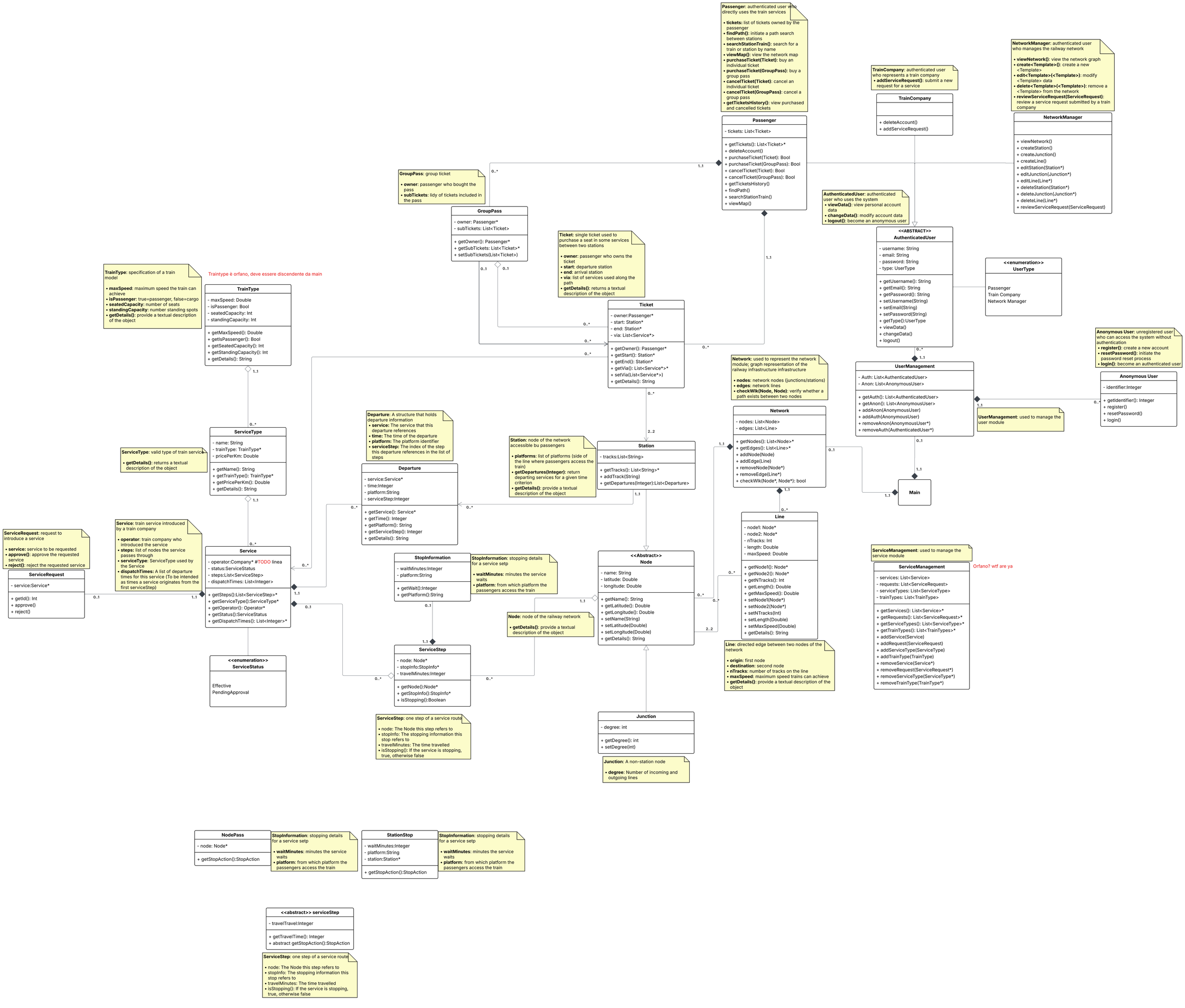
Passenger: authenticated user who directly uses the train services
• tickets: list of tickets owned by the passenger
• findPath(): initiate a path search between stations
• searchStationTrain(): search for a train or station by name
• viewMap(): view the network map
• purchaseTicket(Ticket): buy an individual ticket
• purchaseTicket(GroupPass): buy a group pass
• cancelTicket(Ticket): cancel an individual ticket
• cancelTicket(GroupPass): cancel a group pass
• getTicketsHistory(): view purchased and cancelled tickets
Passenger
- tickets: List<Ticket>
+ getTickets(): List<Ticket>*
+ deleteAccount()
+ purchaseTicket(Ticket): Bool
+ purchaseTicket(GroupPass): Bool
+ cancelTicket(Ticket): Bool
+ cancelTicket(GroupPass): Bool
+ getTicketsHistory()
+ findPath()
+ searchStationTrain()
+ viewMap()
TrainCompany: authenticated user who represents a train company
• addServiceRequest(): submit a new request for a service
TrainCompany
+ deleteAccount()
+ addServiceRequest()
NetworkManager: authenticated user who manages the railway network
• viewNetwork(): view the network graph
• create<Template>(): create a new <Template>
• edit<Template>(<Template>): modify <Template> data
• delete<Template>(<Template>): remove a <Template> from the network
• reviewServiceRequest(ServiceRequest): review a service request submitted by a train company
NetworkManager
+ viewNetwork()
+ createStation()
+ createJunction()
+ createLine()
+ editStation(Station*)
+ editJunction(Junction*)
+ editLine(Line*)
+ deleteStation(Station*)
+ deleteJunction(Junction*)
+ deleteLine(Line*)
+ reviewServiceRequest(ServiceRequest)
GroupPass: group ticket
• owner: passenger who bought the pass
• subTickets: lidy of tickets included in the pass
GroupPass
- owner: Passenger*
- subTickets: List<Ticket>
+ getOwner(): Passenger*
+ getSubTickets: List<Ticket>*
+ setSubTickets(List<Ticket>)
AuthenticatedUser: authenticated user who uses the system
• viewData(): view personal account data
• changeData(): modify account data
• logout(): become an anonymous user
<<ABSTRACT>>
AuthenticatedUser
- username: String
- email: String
- password: String
- type: UserType
+ getUsername(): String
+ getEmail(): String
+ getPassword(): String
+ setUsername(String)
+ setEmail(String)
+ setPassword(String)
+ getType():UserType
+ viewData()
+ changeData()
+ logout()
<<enumeration>>
UserType
Passenger
Train Company
Network Manager
Ticket: single ticket used to purchase a seat in some services between two stations
• owner: passenger who owns the ticket
• start: departure station
• end: arrival station
• via: list of services used along the path
• getDetails(): returns a textual description of the object
Ticket
- owner:Passenger*
- start: Station*
- end: Station*
- via: List<Service*>
+ getOwner(): Passenger*
+ getStart(): Station*
+ getEnd(): Station*
+ getVia(): List<Service*>*
+ setVia(List<Service*>)
+ getDetails(): String
TrainType: specification of a train model
• maxSpeed: maximum speed the train can achieve
• isPassenger: true=passenger, false=cargo
• seatedCapacity: number of seats
• standingCapacity: number standing spots
• getDetails(): provide a textual description of the object
Traintype è orfano, deve essere discendente da main
TrainType
- maxSpeed: Double
- isPassenger: Bool
- seatedCapacity: Int
- standingCapacity: Int
+ getMaxSpeed(): Double
+ getIsPassenger(): Bool
+ getSeatedCapacity(): Int
+ getStandingCapacity(): Int
+ getDetails(): String
Network: used to represent the network module; graph representation of the railway infrastructure infrastructure
• nodes: network nodes (junctions/stations)
• edges: network lines
• checkWlk(Node, Node): verify whether a path exists between two nodes
Anonymous User: unregistered user who can access the system without authentication
• register(): create a new account
• resetPassword(): initiate the password reset process
• login(): become an authenticated user
UserManagement
- Auth: List<AuthenticatedUser>
- Anon: List<AnonymousUser>
+ getAuth(): List<AuthenticatedUser>
+ getAnon(): List<AnonymousUser>
+ addAnon(AnonymousUser)
+ addAuth(AnonymousUser)
+ removeAnon(AnonymousUser*)
+ removeAuth(AuthenticatedUser*)
Anonymous User
- identifier:Integer
+ getIdentifier(): Integer
+ register()
+ resetPassword()
+ login()
UserManagement: used to manage the user module
Departure: A structure that holds departure information
• service: The service that this departure references
• time: The time of the departure
• platform: The platform identifier
• serviceStep: The index of the step this departure references in the list of steps
Station: node of the network accessible bu passengers
• platforms: list of platforms (side of the line where passengers access the train)
• getDepartures(Integer): return departing services for a given time criterion
• getDetails(): provide a textual description of the object
Station
- tracks:List<String>
+ getTracks(): List<String>*
+ addTrack(String)
+ getDepartures(Integer):List<Departure>
Network
- nodes: List<Node>
- edges: List<Line>
+ getNodes(): List<Node>*
+ getEdges(): List<Line>*
+ addNode(Node)
+ addEdge(Line)
+ removeNode(Node*)
+ removeEdge(Line*)
+ checkWlk(Node*, Node*): bool
Main
ServiceType
- name: String
- trainType: TrainType*
- pricePerKm: Double
+ getName(): String
+ getTrainType(): TrainType*
+ getPricePerKm(): Double
+ getDetails(): String
ServiceType: valid type of train service
• getDetails(): returns a textual description of the object
Departure
- service:Service*
- time:Integer
- platform:String
- serviceStep:Integer
+ getService(): Service*
+ getTime(): Integer
+ getPlatform(): String
+ getServiceStep(): Integer
+ getDetails(): String
Line
- node1: Node*
- node2: Node*
- nTracks: Int
- length: Double
- maxSpeed: Double
+ getNode1(): Node*
+ getNode2(): Node*
+ getNTracks(): Int
+ getLength(): Double
+ getMaxSpeed(): Double
+ setNode1(Node*)
+ setNode2(Node*)
+ setNTracks(Int)
+ setLength(Double)
+ setMaxSpeed(Double)
+ getDetails(): String
ServiceRequest: request to introduce a service
• service: service to be requested
• approve(): approve the requested service
• reject(): reject the requested service
Service: train service introduced by a train company
• operator: train company who introduced the service
• steps: list of nodes the service passes through
• serviceType: ServiceType used by the Service
• dispatchTimes: A list of departure times for this service (To be intended as times a service originates from the first serviceStep)
ServiceRequest
- service:Service*
+ getId(): Int
+ approve()
+ reject()
Service
- operator:Company* #TODO linea
- status:ServiceStatus
- steps:List<ServiceStep>
- dispatchTimes: List<Integer>
+ getSteps():List<ServiceStep>*
+ getServiceType():ServiceType*
+ getOperator(): Operator*
+ getStatus():ServiceStatus
+ getDispatchTimes(): List<Integer>*
StopInformation
- waitMinutes:Integer
- platform:String
+ getWait():Integer
+ getPlatform():String
StopInformation: stopping details for a service setp
• waitMinutes: minutes the service waits
• platform: from which platform the passengers access the train
<<Abstract>>
Node
- name: String
- latitude: Double
- longitude: Double
+ getName(): String
+ getLatitude(): Double
+ getLongitude(): Double
+ setName(String)
+ setLatitude(Double)
+ setLongitude(Double)
+ getDetails(): String
ServiceManagement: used to manage the service module
ServiceManagement
Orfano? wtf are ya
- services: List<Service>
- requests: List<ServiceRequest>
- serviceTypes: List<ServiceType>
- trainTypes: List<TrainType>
+ getServices(): List<Service>*
+ getRequests(): List<ServiceRequest>*
+ getServiceTypes(): List<ServiceType>*
+ getTrainTypes(): List<TrainTypes>*
+ addService(Service)
+ addRequest(ServiceRequest)
+ addServiceType(ServiceType)
+ addTrainType(TrainType)
+ removeService(Service*)
+ removeRequest(ServiceRequest*)
+ removeServiceType(ServiceType*)
+ removeTrainType(TrainType*)
<<enumeration>>
ServiceStatus
Effective
PendingApproval
Node: node of the railway network
• getDetails(): provide a textual description of the object
ServiceStep
- node: Node*
- stopInfo:StopInfo*
- travelMinutes:Integer
+ getNode():Node*
+ getStopInfo():StopInfo*
+ isStopping():Boolean
ServiceStep: one step of a service route
• node: The Node this step refers to
• stopInfo: The stopping information this stop refers to
• travelMinutes: The time travelled
• isStopping(): If the service is stopping, true, otherwise false
Line: directed edge between two nodes of the network
• origin: first node
• destination: second node
• nTracks: number of tracks on the line
• maxSpeed: maximum speed trains can achieve
• getDetails(): provide a textual description of the object
Junction
- degree: int
+ getDegree(): int
+ setDegree(int)
Junction: A non-station node
• degree: Number of incoming and outgoing lines
NodePass
- node: Node*
+ getStopAction():StopAction
StopInformation: stopping details for a service setp
• waitMinutes: minutes the service waits
• platform: from which platform the passengers access the train
StationStop
- waitMinutes:Integer
- platform:String
- station:Station*
+ getStopAction():StopAction
StopInformation: stopping details for a service setp
• waitMinutes: minutes the service waits
• platform: from which platform the passengers access the train
<<abstract>> serviceStep
- travelTravel:Integer
+ getTravelTime(): Integer
+ abstract getStopAction():StopAction
ServiceStep: one step of a service route
• node: The Node this step refers to
• stopInfo: The stopping information this stop refers to
• travelMinutes: The time travelled
• isStopping(): If the service is stopping, true, otherwise false